# Practical Project Day
# Building Your MCP Server

Design, implement, test, and demonstrate a custom MCP server in groups of 2–3.

| | |
|---|---|
| Format | Group project (2–3 students) |
| Stack | Python · FastMCP · Claude Desktop / Gemini CLI |
| Deliverables | Working server + documentation + live demo |

Yesterday

# You learned how agents work

ReAct loop → Prompt design → MCP protocol → First server

Today

# You build one from scratch

# Day 2 — Roadmap

From design to demonstration in 6 hours

| H1 | H2 | H3 | H4 | H5 | H6 |
|---|---|---|---|---|---|
| Design | Sprint 1 | Sprint 2 | Testing | Analysis | Demos |

| Hour | Phase | Format | Deliverable |
|---|---|---|---|
| 1 | Project Kickoff + Server Design | LECTURE GROUP | Checked-off design document |
| 2 | Build Sprint 1 | SPRINT | Server with 2 tools in Inspector |
| 3 | Build Sprint 2 + Integration | SPRINT HANDS-ON | Full server connected to Claude Desktop / Gemini CLI |
| 4 | Testing + Prompting Strategies | DEMO HANDS-ON | Working test scenarios with results |
| 5 | Comparative Analysis + Documentation | HANDS-ON | README + comparison analysis |
| 6 | Demonstrations + Debrief | PRESENT | Final presentations + course synthesis |

HOUR 1

# Project Kickoff + Server Design

60 minutes · Briefing + Group Formation + Guided Design

# 1.1 — Choose Your Project

15 min · Each group picks one · Duplicates are fine

PROJECT A

## Academic Research Assistant

**Do a 2-hour literature review in 30 seconds.**

Search arXiv, retrieve paper metadata, compare research on a topic.

| | |
|---|---|
| Difficulty | Medium |
| Libraries | `requests`, `xml.etree` |
| Challenge | Search → retrieve → compare pipeline |

PROJECT B

## Data Query Builder

**Ask questions about data in English instead of writing SQL.**

Manage a SQLite database, load CSVs, run queries, compute statistics.

| | |
|---|---|
| Difficulty | Medium |
| Libraries | `sqlite3`, `csv` (stdlib) |
| Challenge | Natural-language questions → SQL queries + result formatting |

PROJECT C

## Debugging Assistant

**Find why a system crashed without reading 10,000 lines of logs.**

Ingest log files, detect error patterns, correlate events.

| | |
|---|---|
| Difficulty | Medium-Hard |
| Libraries | `re`, `json` (stdlib) |
| Challenge | Pattern detection + cross-file correlation |

# Rules for All Projects

### REQUIREMENTS

- At least **3 tools** and **1 resource**
- Every tool has **typed parameters** (Python type hints)
- Every tool has a **clear docstring**
- Handle errors gracefully — return messages, don't crash

### DOCUMENTATION

- README with setup instructions
- Tool descriptions table
- One usage example per tool
- With/without tools comparison

### REMEMBER FROM DAY 1

The **docstring is the tool description** — it's the only thing Claude reads to decide when and how to use your tool. Write it like you're explaining the tool to a person. Vague descriptions = misused tools.

### HOW YOU'LL BE EVALUATED

**Tool Design** (25%) + **Implementation** (25%) + **Claude Desktop / Gemini CLI Integration** (20%) + **Documentation** (15%) + **Comparison Analysis** (15%). Start collecting comparison data early — you'll need it for the final presentation.

PROJECT A

# Academic Research Assistant — Tool Spec

| Tool | Description | Key Parameters |
|---|---|---|
| `search_papers` | Search arXiv by query, optional category and max results | `query: str`, `category: str = ""`, `max_results: int = 5` |
| `get_paper_details` | Fetch full metadata for a paper by arXiv ID | `arxiv_id: str` |
| `search_by_author` | Find papers by a specific author | `author_name: str`, `max_results: int = 5` |
| `get_related_papers` | Find papers sharing categories with a given paper | `arxiv_id: str`, `max_results: int = 5` |
| `compare_papers` | Structured comparison of 2–3 papers | `arxiv_ids: list[str]` |

**RESOURCE**

`arxiv://search-history`

JSON list of all searches performed this session

**RATE LIMIT**

arXiv API: max 1 request per 3 seconds. Use `time.sleep(3)` between calls. The provided helper handles this.

PROJECT A

# Pre-written arXiv Helper

This is **provided to you**. You don't write XML parsing — you focus on tool design.

```python
"""arXiv API helper — handles HTTP requests and XML parsing."""
import time
import requests
import xml.etree.ElementTree as ET

ARXIV_API = "http://export.arxiv.org/api/query"
ATOM_NS = "{http://www.w3.org/2005/Atom}"
_last_request_time = 0

def _rate_limit():
    global _last_request_time
    elapsed = time.time() - _last_request_time
    if elapsed < 3:
        time.sleep(3 - elapsed)
    _last_request_time = time.time()

def search_arxiv(query: str, max_results: int = 5) -> list[dict]:
    """Search arXiv. Returns list of dicts:
    id, title, authors, abstract, published, categories, pdf_url"""
    _rate_limit()
    params = {"search_query": query, "start": 0, "max_results": max_results}
    resp = requests.get(ARXIV_API, params=params, timeout=15)
    resp.raise_for_status()
    root = ET.fromstring(resp.text)
    results = []
    for entry in root.findall(f"{ATOM_NS}entry"):
        paper = {
            "id": entry.findtext(f"{ATOM_NS}id", "").split("/abs/")[-1],
            "title": " ".join(entry.findtext(f"{ATOM_NS}title", "").split()),
            "authors": [a.findtext(f"{ATOM_NS}name", "")
                        for a in entry.findall(f"{ATOM_NS}author")],
            "abstract": " ".join(entry.findtext(f"{ATOM_NS}summary", "").split()),
            "published": entry.findtext(f"{ATOM_NS}published", "")[:10],
            "categories": [c.get("term", "")
                           for c in entry.findall(f"{ATOM_NS}category")],
            "pdf_url": "",
        }
        for link in entry.findall(f"{ATOM_NS}link"):
            if link.get("title") == "pdf":
                paper["pdf_url"] = link.get("href", "")
        results.append(paper)
    return results

def get_paper_by_id(arxiv_id: str) -> dict | None:
    """Fetch a single paper by its arXiv ID (e.g. '2210.03629').
    Returns a dict with the same keys as search_arxiv, or None."""
```

```
    results = search_arxiv(f"id:{arxiv_id}", max_results=1)
    return results[0] if results else None
```

Use `search_arxiv()` and `get_paper_by_id()` inside your tools. Focus on tool design, not HTTP/XML plumbing.

PROJECT B

# Data Query Builder — Tool Spec

| Tool | Description | Key Parameters |
|---|---|---|
| `load_csv` | Load a CSV file into a new SQLite table with auto-detected types | `file_path: str`, `table_name: str` |
| `describe_schema` | List all tables and their columns/types | *(none)* |
| `run_query` | Execute a **read-only** SQL query and return results | `sql: str`, `limit: int = 50` |
| `get_statistics` | Summary stats for a column: count, min, max, mean, nulls | `table_name: str`, `column: str` |
| `list_tables` | List all tables with their row counts | *(none)* |

**RESOURCES**

`db://schema` — Current database schema as JSON

`db://query-history` — Queries executed this session

**SECURITY**

`run_query` must **reject** queries containing `DROP`, `DELETE`, `ALTER`, `INSERT`, `UPDATE`. Default to read-only.

PROJECT B

# Pre-written SQLite Helper

```python
"""SQLite helper — database setup and CSV loading."""
import csv, sqlite3

def create_db() -> sqlite3.Connection:
    """Create an in-memory SQLite database."""
    conn = sqlite3.connect(":memory:")
    conn.row_factory = sqlite3.Row
    return conn

def _is_int(v: str) -> bool:
    try: int(v); return True
    except ValueError: return False

def _is_float(v: str) -> bool:
    try: float(v); return True
    except ValueError: return False

def _cast(value: str, col_type: str):
    """Cast a CSV string value to the detected column type."""
    if not value:
        return None
    if col_type == "INTEGER": return int(value)
    if col_type == "REAL":    return float(value)
    return value

def load_csv_to_table(conn, file_path: str, table_name: str) -> dict:
    """Load a CSV into a SQLite table. Auto-detects column types.
    Returns: {"table_name": str, "columns": [...], "row_count": int}"""
    with open(file_path, "r") as f:
        reader = csv.DictReader(f)
        rows = list(reader)
    if not rows:
        raise ValueError(f"CSV file is empty: {file_path}")

    # Auto-detect types from first 100 rows
    columns = {}
    for col in rows[0].keys():
        sample = [r[col] for r in rows[:100] if r[col]]
        if all(_is_int(v) for v in sample):
            columns[col] = "INTEGER"
        elif all(_is_float(v) for v in sample):
            columns[col] = "REAL"
        else:
            columns[col] = "TEXT"

    col_defs = ", ".join(f'"{col}" {typ}' for col, typ in columns.items())
    conn.execute(f'CREATE TABLE IF NOT EXISTS "{table_name}" ({col_defs})')
    placeholders = ", ".join("?" for _ in columns)
    col_names = ", ".join(f'"{c}"' for c in columns)
    for row in rows:
```

```
        values = [_cast(row[c], columns[c]) for c in columns]
        conn.execute(f'INSERT INTO "{table_name}" ({col_names}) VALUES ({placeholders})', values)
    conn.commit()
    return {"table_name": table_name, "columns": list(columns.items()), "row_count": len(rows)}
```

PROJECT C

# Debugging Assistant — Tool Spec

| Tool | Description | Key Parameters |
|---|---|---|
| `ingest_log` | Load a log file, auto-detect format (JSONL, syslog, plaintext) | `file_path: str`, `format: str = "auto"` |
| `search_logs` | Search loaded logs by keyword, level, or time range | `query: str = ""`, `level: str = ""`, `start_time: str = ""`, `end_time: str = ""` |
| `get_error_summary` | Group errors by message pattern, count occurrences | *(none)* |
| `detect_anomalies` | Find time windows with unusually high error rates | `window_minutes: int = 5` |
| `correlate_events` | Find all entries within ±N seconds of a timestamp across files | `timestamp: str`, `window_seconds: int = 30` |

**RESOURCE**

`logs://files`

List of loaded log files with metadata (path, format, line count, time range)

**DATA MODEL**

Store parsed entries as: `{"timestamp", "level", "message", "raw", "file", "line"}`

PROJECT C

# Pre-written Log Parser Helper

```python
"""Log format detection and parsing helpers."""
import json, re
from datetime import datetime

SYSLOG_PATTERN = re.compile(
    r"^(\w{3}\s+\d{1,2}\s+\d{2}:\d{2}:\d{2})\s+(\S+)\s+(\S+?):\s*(.*)"
)
TIMESTAMP_PATTERN = re.compile(
    r"^(\d{4}-\d{2}-\d{2}[T ]\d{2}:\d{2}:\d{2})"
)
# For normalizing error messages (replace variable parts)
NORMALIZE_PATTERNS = [
    (re.compile(r"\b[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}\b"), "<UUID>"),
    (re.compile(r"\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b"), "<IP>"),
    (re.compile(r"\b\d+\b"), "<NUM>"),
]

def detect_format(lines: list[str]) -> str:
    """Detect log format from first few lines."""
    sample = [l.strip() for l in lines[:5] if l.strip()]
    if not sample:
        return "plaintext"
    if all(l.startswith("{") for l in sample):
        return "jsonl"
    if all(SYSLOG_PATTERN.match(l) for l in sample):
        return "syslog"
    return "plaintext"

def normalize_message(msg: str) -> str:
    """Replace variable parts with placeholders for grouping."""
    for pattern, replacement in NORMALIZE_PATTERNS:
        msg = pattern.sub(replacement, msg)
    return msg
```

# 1.4 — Guided Design Phase

35 min · GROUP WORK · On paper or whiteboard

Each group produces a **design document** covering:

1. **Tool inventory**
   Name, one-line description, parameters, return format
2. **Resource inventory**
   URI and what it exposes
3. **Data model**
   What information does your server remember while running?
4. **Division of labor**
   Who implements what?
5. **Error cases**
   What if the API is down / file missing / query invalid?

COMMON DESIGN MISTAKES (INSTRUCTOR WILL CHECK)

- **Tool does too much** — a single `analyze_everything` tool → break it down
- **Tool does too little** — separate tools for trivially different things → combine
- **No error handling** — what if `file_path` doesn't exist?
- **Vague docstrings** — "Does stuff with data" → rewrite to be specific

GATE

Design document must be **checked off by instructor** before coding starts.

HOUR 2

# Build Sprint 1

60 minutes · Scaffolding + First 2 Tools

**Goal: server with 2 working tools in Inspector by end of hour**

# 2.1 — Scaffolding

10 min · All groups in parallel

Start from the Day 1 skeleton:

```
mkdir project-name && cd project-name
uv venv && source .venv/bin/activate
uv pip install "mcp[cli]"

# Project-specific dependencies:
# A: uv pip install requests
# B: (none — sqlite3 and csv are stdlib)
# C: (none — json and re are stdlib)
```

Copy Day 1 `server.py` → Copy project helper file → Replace `hello` + `add` with real tools

**COMMON BLOCKERS DURING SPRINT**

| | |
|---|---|
| arXiv returns unexpected XML | Use the provided `search_arxiv()`, don't parse manually |
| `sqlite3.OperationalError` | Wrap column names in double quotes: `"Column Name"` |
| Log encoding issues | `open(path, "r", errors="replace")` |
| `list[str]` not working | Python ≥ 3.10 required. On 3.9: `from typing import List` |
| `mcp dev` fails | Check `mcp[cli]` is installed in the active venv |

**CHECKPOINT — END OF HOUR 2**

You should have **2 tools working in MCP Inspector** (`mcp dev server.py`). If you're not there yet, call the instructor — don't move on with broken tools.

HOUR 3

# Build Sprint 2 + Claude Desktop / Gemini CLI Integration

60 minutes · Complete tools + Connect to Claude

# 3.1 — Sprint Continuation

40 min · SPRINT

Complete remaining tools + at least one resource.

TEST DATA — CREATE BEFORE TESTING

### Project B — sample CSV

```
name,region,category,price,quantity
Widget A,North,Electronics,29.99,150
Widget B,South,Electronics,49.99,80
Gadget C,North,Home,15.50,300
Gadget D,East,Home,22.00,120
Service E,South,Software,99.00,45
```

Save as `sample_data.csv` in your project folder.

### Project C — sample log

```
2025-01-15T14:12:01 INFO  Server started on port 8080
2025-01-15T14:13:22 WARN  Slow query: 2.3s on /api/users
2025-01-15T14:15:01 ERROR DB connection timeout
2025-01-15T14:15:02 ERROR Failed to process request
2025-01-15T14:15:03 ERROR DB connection timeout
2025-01-15T14:16:10 INFO  DB connection restored
```

Save as `sample.log`. Add more lines to test patterns.

CHALLENGES FOR GROUPS AHEAD OF SCHEDULE

- Add a tool whose output **depends on another tool's prior results** (tool composition)
- Add input validation that returns **helpful error messages**

  e.g., "arXiv ID should look like 2301.12345, you passed: xyz"
- Make a resource that **updates dynamically** as tools are called (e.g., search history)

# 3.2 — Claude Desktop / Gemini CLI Integration

20 min · HANDS-ON

## 1 Update your config file

Claude Desktop: `claude_desktop_config.json` — Gemini CLI: `~/.gemini/settings.json`

```
{
  "mcpServers": {
    "project-name": {
      "command": "/absolute/path/to/.venv/bin/python",
      "args": ["/absolute/path/to/server.py"]
    }
  }
}
```

## 2 Restart Claude Desktop or relaunch Gemini CLI

## 3 Test with a natural language prompt

**Project A**
"Search for recent papers on transformer architectures in cs.AI"

**Project B**
"Load this CSV file and tell me the average sales by region"

**Project C**
"Load this log file and show me a summary of all errors"

### TROUBLESHOOTING CHECKLIST

- ☐ Server runs standalone? (`python server.py` prints nothing to stdout)
- ☐ Inspector works? (`mcp dev server.py`)
- ☐ Path in config is **absolute**?
- ☐ Using the **venv Python**, not system Python?
- ☐ **Restarted** Claude Desktop / relaunched Gemini CLI after config change?

### CHECKPOINT — END OF HOUR 3

Your server should be **connected to Claude Desktop or Gemini CLI and responding to natural language prompts**. If the AI doesn't see your tools, work through the checklist above before moving on.

HOUR 4

# Testing + Prompting Strategies

60 minutes · Demo + Lecture + Group Work

HOUR 4

# 4.1 — The Programmatic Agent Loop

10 min · INSTRUCTOR DEMO

What's happening under the hood of Claude Desktop / Gemini CLI:

```
The Agent Loop — 3 steps, repeated

┌──────────────┐          ┌──────────────┐          ┌──────────────┐
│ 1. ASK       │          │ 2. DECIDE    │          │ 3. EXECUTE   │
│ Send message │   →      │ Model picks  │   →      │ Run the tool │
│ + tools list │          │ a tool (or   │          │ and feed the │
│ to Claude    │          │ answers)     │          │ result back  │
└──────────────┘          └──────────────┘          └──────────────┘
                                                          │
                                                          │
                          Loop until the model says "I'm done"  ←┘
```

```python
"""Minimal agent loop — this is what Claude Desktop / Gemini CLI does."""
import anthropic

client = anthropic.Anthropic()   # uses ANTHROPIC_API_KEY

# ── Tool declaration: tell Claude what tools exist ──────────
tools = [{
    "name": "search_papers",
    "description": "Search arXiv for papers matching a query.",
    "input_schema": {
        "type": "object",
        "properties": {"query": {"type": "string"}},
        "required": ["query"],
    },
}]

messages = [{"role": "user", "content": "Find papers about RLHF"}]

while True:
    # ── STEP 1: Ask Claude (send message + available tools) ─
    response = client.messages.create(
        model="claude-sonnet-4-5-20250514",
        max_tokens=4096, tools=tools, messages=messages,
    )
    messages.append({"role": "assistant", "content": response.content})

    # ── STEP 2: Did Claude decide to answer or use a tool? ──
    if response.stop_reason == "end_turn":       # ← "I'm done"
        for block in response.content:
```

```
        if block.type == "text": print(block.text)
    break

# ── STEP 3: Execute tool calls and feed results back ───
tool_results = []
for block in response.content:
    if block.type == "tool_use":
        result = execute_mcp_tool(block.name, block.input)  # ← calls your MCP server
        tool_results.append({
            "type": "tool_result",
            "tool_use_id": block.id,
            "content": result,
        })
messages.append({"role": "user", "content": tool_results})  # ← back to step 1
```

**KEY INSIGHT**

This is the **ReAct loop from Day 1 Hour 1**, now concrete Python. The model decides which tools to call and when to stop. The loop is mechanical — the intelligence is in the model. **That's why tool descriptions matter.**

HOUR 4

# 4.2 — Three Prompting Strategies

15 min · LECTURE

### Strategy 1: Minimal

```
You have access to tools.
Use them to help the user.
```

No guidance. Model figures it out.

### Strategy 2: Structured

```
You are a [domain] assistant.

Before using any tool:
1. State what you're trying
   to learn
2. Choose the best tool
3. Explain what you learned

If a tool returns an error,
explain and try alternatives.
```

Step-by-step reasoning guidance.

### Strategy 3: Expert Workflow

```
You are a senior [domain] expert.
Work methodically:

Phase 1 — Reconnaissance:
  Use [tool_a] to understand scope.
Phase 2 — Deep analysis:
  Use [tool_b] on specific items.
Phase 3 — Synthesis:
  Combine into recommendations.

Cite which tool provided each
finding.
```

Explicit workflow phases.

**CONCRETE EXAMPLE — STRATEGY 3 FOR PROJECT A**

```
You are a senior research analyst.
Work methodically:

Phase 1 — Discovery:
  Use search_papers to find relevant work on the topic.
Phase 2 — Deep dive:
  Use get_paper_details on the most promising results.
Phase 3 — Synthesis:
  Use compare_papers to produce a structured comparison.

Cite which tool provided each finding.
```

Adapt this template to your own project. Replace tool names and phases with yours.

**KEY POINT**

Strategy 3 significantly outperforms Strategy 1 on complex tasks because it gives the model a **decomposition plan**. This mirrors Claude Code's prompt design from Day 1: explicit workflow phases produce better tool use sequences. **You will test this empirically next.**

# 4.3 — Fix Bugs + Design Test Scenarios

35 min · GROUP WORK

| Fix bugs (10 min) | System prompt (5 min) | Test scenarios (5 min) | Run scenarios in Claude Desktop / Gemini CLI (15 min) |
|---|---|---|---|

## Define 3 Test Scenarios Requiring Multi-Tool Orchestration

| | Scenario | Expected Tool Sequence |
|---|---|---|
| A | "Find papers on RLHF from 2023 and compare the top 3" | `search_papers` → `get_paper_details` ×3 → `compare_papers` |
| A | "What has Yann LeCun published recently?" | `search_by_author` → `get_paper_details` |
| B | "Load sales data and find highest revenue region by quarter" | `load_csv` → `describe_schema` → `run_query` ×N |
| B | "Average price per product category?" | `load_csv` → `run_query` → `get_statistics` |
| C | "What caused the error spike at 14:15?" | `ingest_log` ×2 → `search_logs` → `correlate_events` |
| C | "Are there patterns in the last hour's errors?" | `ingest_log` → `detect_anomalies` → `search_logs` |

Record what happens: which tools were called, in what order, whether the result was useful.

HOUR 5

# Comparative Analysis + Documentation

60 minutes · Measure the difference

# 5.1 — With vs. Without Tools

20 min · GROUP WORK

Run the **same complex task twice**:

### Without MCP Tools

Fresh Claude conversation. No MCP servers connected. Ask your test scenario. Record the response.

**Projects B/C:** paste a small data sample directly in your prompt so Claude has something to work with — this compares "raw data in context" vs. "structured tools."

### With MCP Tools

Identical question with your server connected. Record the response.

## Comparison Dimensions

| Dimension | Without Tools | With Tools |
|---|---|---|
| **Accuracy** | Does Claude hallucinate facts? | Does it use real data from tools? |
| **Specificity** | Generic response? | Specific to actual data? |
| **Completeness** | Missing information? | Can it gather what it needs? |
| **Confidence** | Hedging and qualifying? | Citing tool results? |
| **Latency** | One fast response | Multiple tool call round-trips |

# 5.2 — Prompting Strategy Comparison

15 min · GROUP WORK

Run the same complex scenario **twice** in Claude Desktop or Gemini CLI. We compare the extremes — Strategy 2 is a middle ground you can explore on your own.

**With Strategy 1 (Minimal)**

*"You have access to tools. Use them."*

**With Strategy 3 (Expert Workflow)**

Your custom system prompt with phases and cited findings.

## What to Document

- How many **tool calls** each strategy triggered
- Did Claude **plan before acting** (Strategy 3) vs. dive in (Strategy 1)?
- **Quality of the final synthesis**
- Any **errors or dead ends**

**THIS IS PART OF YOUR FINAL PRESENTATION**

Show concrete examples of how prompting strategy affected tool usage quality. Screenshots are great.

# 5.3 — Documentation Sprint

25 min · Write your `README.md`

```
# [Server Name]

## Overview
One paragraph: what this server does and why.

## Setup
```bash
uv venv && source .venv/bin/activate
uv pip install "mcp[cli]"         # + project-specific deps
```

## Claude Desktop / Gemini CLI Configuration
```json
{ "mcpServers": { "name": { "command": "...", "args": ["..."] } } }
```

## Tools

### `tool_name`
**Description:** What it does.
**Parameters:** | param | type | required | description |
**Example:** "Search for papers about RL" → calls search_papers

## Resources
### `resource://uri` — What it provides.

## Limitations
## Comparison Results
[Paste the with/without tools comparison table]
[Paste the prompting strategy comparison]
```

HOUR 6

# Demonstrations + Debrief

60 minutes · Show what you built

HOUR 6

# 6.1 — Group Demonstrations

35 min · ~10 min per group

| Overview (2 min) | Live Demo (5 min) | Comparison (2 min) | Retro (1 min) |
|---|---|---|---|

1. **Server overview** (2 min) — What you built, what tools/resources it exposes
2. **Live demo** (5 min) — Run your most complex test scenario live in Claude Desktop or Gemini CLI. Show multi-tool orchestration in action.
3. **Comparison results** (2 min) — Show with/without tools + prompting strategy comparison
4. **One thing you'd change** (1 min) — What design decision would you revisit?

# Evaluation Criteria

| Criterion | Weight | What We Look For |
|---|---|---|
| Tool Design Quality | 25% | Clear boundaries, specific docstrings, appropriate granularity |
| Implementation Correctness | 25% | Tools work, errors handled gracefully, no crashes |
| Claude Desktop / Gemini CLI Integration | 20% | Multi-tool workflows succeed, Claude uses tools appropriately |
| Documentation | 15% | README is complete, examples work |
| Comparison Analysis | 15% | Thoughtful with/without analysis, prompting strategy insights |

# 6.2 — Course Synthesis

10 min

| Day 1 (Theory) | | Day 2 (Practice) |
|---|---|---|
| Agent loop (ReAct) | → | Saw the loop in real code |
| Claude Code prompt design | → | Wrote system prompts for your server |
| MCP protocol + architecture | → | Built an MCP server from scratch |
| Existing server code reading | → | Reused patterns in your own server |

## What You Can Do Next

### This week

1. Install your server on your personal Claude Desktop or Gemini CLI
2. Extend your server with more tools
3. Explore the MCP server registry to discover existing tools

### Going further

4. Combine multiple MCP servers in Claude Desktop or Gemini CLI
5. Publish your server as a pip-installable package
6. Add HTTP transport for remote deployment

# 6.3 — Finished Early? What to Build Next

INSPIRATION · Real-world MCP server ideas

If you've completed your project ahead of time, here are **industry-grade MCP server ideas** you can explore on your own:

1. **CI/CD Pipeline Monitor**

   Connect to GitHub Actions or GitLab CI APIs. Tools: `get_pipeline_status`, `list_failed_jobs`, `retry_job`, `get_build_logs`. Let the AI diagnose build failures and suggest fixes from log output.

2. **Database Schema Explorer & Query Assistant**

   Connect to a PostgreSQL or MySQL database. Tools: `list_tables`, `describe_table`, `run_readonly_query`, `explain_query_plan`. The AI can explore schemas, write optimized queries, and identify performance bottlenecks — all with read-only safety.

3. **Cloud Infrastructure Dashboard**

   Wrap AWS/GCP/Azure SDK calls. Tools: `list_instances`, `get_instance_metrics`, `check_security_groups`, `estimate_monthly_cost`. Give the AI visibility into your cloud resources for cost optimization and security auditing.

4. **Customer Support Ticket Triage**

   Integrate with Zendesk, Jira, or a ticket database. Tools: `search_tickets`, `get_ticket_details`, `classify_priority`, `suggest_response`, `link_related_tickets`. Automate ticket categorization and draft responses based on historical resolutions.

5. **Internal Knowledge Base Search**

   Index Confluence, Notion, or internal docs. Tools: `search_docs`, `get_page_content`, `list_recent_updates`, `find_subject_expert`. Turn scattered company documentation into an AI-searchable knowledge base that surfaces relevant answers instantly.

**WHY THESE IDEAS?**

Each one follows the same patterns you learned today: **clear tool boundaries**, **specific input schemas**, and **security constraints** (read-only access, allowed directories, API scoping). The MCP architecture is the same — only the domain changes.

# Reference Links

| Resource | URL |
| --- | --- |
| MCP Server Registry | `modelcontextprotocol.io/registry` |
| MCP Python SDK | `github.com/modelcontextprotocol/python-sdk` |
| Anthropic Tool Use Docs | `docs.anthropic.com/en/docs/build-with-claude/tool-use` |
| arXiv API Manual | `info.arxiv.org/help/api/user-manual.html` |
| Python sqlite3 Docs | `docs.python.org/3/library/sqlite3.html` |

# Workshop Complete

You've designed, built, tested, and demonstrated a custom MCP server — and compared how an AI agent performs with and without your tools.

```
12 hours  ·  ReAct → Tool Use → MCP → Custom Server

You are now equipped to extend AI systems with custom tools.
```